



How to choose a database 10m

Finding the right DBMS for your application is not straightforward. The popular and most widely used database types are:

- online transactional processing (OLTP)
- online analytical processing (OLAP)
- hybrid transaction/analytical processing (HTAP)
- SQL
- NoSQL.

Before selecting a database, it is important to ask yourself the following questions. Think about your project when answering these questions.

Question	Description
What are your preferred programming languages?	Your preferred programming language influences your choice of database. Take, for example, if JavaScript is your preferred programming language, then your database might only support JSON data types. Additional software may be required to get your programming language to work with a particular database.
What programming languages will you be using for your web application?	
How much data do you expect to store when the application is mature (over a period of time)?	There are many options for databases that handle the storage of data in gigabytes or less. However, there are significantly fewer databases available that handle the storage of data in petabytes (millions of gigabytes); these databases will also come with high data storage costs.

[Skip to main content](#)

How many users do you expect to handle simultaneously at peak load?	It is important to estimate how many users will use your database because you need to estimate the load on the database from many users simultaneously querying the database.
What is the level of availability, scalability, latency, throughput and data consistency that your application will need?	Availability is important in transactional databases that serve applications such as banks and retail stores. It is important these databases run 24/7 so users can use the application. Databases with good scalability can handle the load of multiple simultaneous users. They do this by scaling up or out until the throughput is sufficient to handle the load. The latency should be good, meaning the database should respond in a short amount of time for each request.
How often will your database schemas change?	If your schemas are likely to change, then a NoSQL database is best. If the schemas are unlikely to change significantly and you want the fields to have consistent types in the records, then an SQL database is best.
What is the geographic location of your user population? Are they distributed?	You need to consider where your users will be located because if your database users are going to be from all over the world, this will affect the database latency for remote users unless you have a server located in their region of the world.
Does your application need online transaction processing (OLTP), analytic queries (OLAP) or both?	If your database needs online transaction processing, then this implies it needs fast write speed and minimal indexes, whereas needing analytic queries implies fast read speed and lots of indexes. If you need both, then you need to use a hybrid system.
What ratio of reads to writes do you expect in production?	Consider the percentage of read and write operations you expect in your application because some databases are faster at reads and queries while others are faster at writes.
What is your budget? If so, will it cover licences and support contracts?	Databases can be free or have paid features or paid versions. Free databases often do not come with support from the vendor, so this needs to be taken into account when thinking about budget.

[Skip to main content](#)

The ratio of reads to write is the percentage of read operations to write operations. **Read** refers to actions that trigger a read operation such as an operation that retrieves data from the database and sends it to the frontend (the data is not changed in any way, making it a read operation). **Write** refers to actions that trigger a write operation that changes the data stored in the database. These are done through actions such as insert, delete and update commands.



Thoughts on this page?

